Lab Assignment 5: Extending xv6 File System

CS-344: Operating Systems Laboratory

Narodia Mit (230123039)
Rehan Sherawat (230123077)
Samir Raj (230123056)
Prakhar Agrawal (230123045)

October 13, 2025

For modified code files: **Google Drive Link**

Contents

1 Task 5.1: Doubly-Indirect Block Support

The default file system in xv6, while effective for teaching, is constrained by a notable file size limitation. The on-disk inode (‘dinode‘) structure allocates 12 direct block pointers and a single-indirect pointer. Given a block size of 1024 bytes and 4-byte block addresses, a single-indirect block can reference $1024/4 = 256$ data blocks. This architecture caps the maximum file size as follows:

$$(12_{\text{direct}} + 256_{\text{single-indirect}}) \times 1024 \text{ bytes} = 268 \times 1024 \text{ bytes} = 268 \text{ KB}$$

This capacity is restrictive for many practical uses.

This task's goal is to expand the file system's capabilities by implementing a **doubly-indirect block pointer**. This requires modifying the inode structure to include the new pointer and adapting the kernel functions responsible for block mapping (‘bmap‘) and file deletion (‘itrunc‘) to manage the extended address space. This enhancement will substantially increase the potential file size, enabling xv6 to handle much larger data sets.

1.1 Kernel Modifications

To enable doubly-indirect block support, modifications were applied to several key areas of the xv6 file system. The changes centered on the inode data structure and the functions governing block management.

1.1.1 Inode Structure Modification (fs.h)

The initial action was to reconfigure the on-disk inode structure, ‘struct dinode‘, located in ‘kernel/fs.h‘. To create space for a doubly-indirect pointer while maintaining the same total number of address pointers (‘NDADDRS‘), the count of direct pointers (‘NDIRECT‘) was decreased from 12 to 11. This adjustment yields a structure with one slot for the direct pointer, one for the single-indirect pointer, and a new one for the doubly-indirect pointer.

```
/* on-disk inode */
#define NDIRECT 11      // reduce from 12 to make room for two extra pointers
#define NINDIRECT (BSIZE / sizeof(uint)) // pointers per indirect block
#define NDADDRS (NDIRECT + 2) // direct + single-indirect + double-indirect
#define IPB (BSIZE / sizeof(struct dinode))
#define MAXFILE ( (NDIRECT + NINDIRECT + (NINDIRECT * NINDIRECT)) * BSIZE )

// On-disk inode structure
struct dinode {
    short type;         // File type
    short major;        // Major device number (T_DEVICE only)
    short minor;        // Minor device number (T_DEVICE only)
    short nlink;        // Number of links to inode in file system
    uint size;          // Size of file (bytes)
    uint addrs[NDADDRS]; // Data block addresses
};
```

Listing 1: Modified inode structure definitions in ‘kernel/fs.h‘.

Following these modifications, the new theoretical maximum file size is derived from:

- **Direct Blocks:** 11×1024 bytes = 11 KB
- **Single-Indirect Blocks:** 256×1024 bytes = 256 KB
- **Doubly-Indirect Blocks:** $256 \times 256 \times 1024$ bytes = 65,536 KB = 64 MB

The new combined maximum file size is now approximately **64.2 MB**.

1.1.2 Block Mapping Logic (‘bmap‘)

The ‘bmap‘ function in ‘kernel/fs.c‘ translates a file’s logical block number into a physical disk block address. This function was enhanced to manage the range covered by the doubly-indirect block.

The updated logic operates sequentially:

1. It first determines if the block number (‘bn‘) is within the direct block range (< NDIRECT).
2. Failing that, it checks if ‘bn‘ falls into the single-indirect range (< NDIRECT+NINDIRECT).
3. If the block number is beyond these ranges, the new logic for doubly-indirect mapping is executed. This involves:
   - Computing the index for the doubly-indirect block and the subsequent index for the intermediate indirect block.
   - Allocating the doubly-indirect block and the necessary indirect block if they haven’t been created yet.
   - Navigating through these pointers to locate the final data block’s address, allocating it if needed.

```
uint bmap(struct inode *ip, uint bn)
{ uint addr, indirect_block, double_indirect_block;
  struct buf *bp, *dbp;
  uint *a;
  uint d_idx, i_idx;

  // direct blocks
  if (bn < NDIRECT) {
    if ((addr = ip->addrs[bn]) == 0) {
      addr = ip->addrs[bn] = balloc(ip->dev);
    }
    return addr;
  }

  // single indirect blocks
  bn -= NDIRECT;
  if (bn < NINDIRECT) {
    // allocate single indirect block if needed
    if ((addr = ip->addrs[NDIRECT]) == 0) {
      addr = ip->addrs[NDIRECT] = balloc(ip->dev);
      // zero it out
      bp = bread(ip->dev, addr);
      memset(bp->data, 0, BSIZE);
      bwrite(bp);
      brelse(bp);
    }
    bp = bread(ip->dev, ip->addrs[NDIRECT]);
    a = (uint*)bp->data;
    if (a[bn] == 0) {
      a[bn] = balloc(ip->dev);
      bwrite(bp);
    }
    addr = a[bn];
    brelse(bp);
    return addr;
  }

  // double indirect blocks
  // adjust bn to start counting after direct + single-indirect range
```

```
  bn -= NINDIRECT;

  // compute indices: which indirect block in the double-indirect, and which
    direct pointer inside it
  d_idx = bn / NINDIRECT; // index inside double-indirect block (which indirect
    -block pointer)
  i_idx = bn % NINDIRECT; // index inside the chosen indirect block

  // allocate double-indirect block if necessary
  if ((double_indirect_block = ip->addrs[NDIRECT + 1]) == 0) {
    double_indirect_block = ip->addrs[NDIRECT + 1] = balloc(ip->dev);
    // zero it out
    dbp = bread(ip->dev, double_indirect_block);
    memset(dbp->data, 0, BSIZE);
    bwrite(dbp);
    brelse(dbp);
  }

  // read the double-indirect block
  dbp = bread(ip->dev, ip->addrs[NDIRECT + 1]);
  a = (uint*)dbp->data;

  // allocate the indirect block pointed to by a[d_idx] if needed
  if (a[d_idx] == 0) {
    a[d_idx] = balloc(ip->dev);
    // zero newly allocated indirect block
    bp = bread(ip->dev, a[d_idx]);
    memset(bp->data, 0, BSIZE);
    bwrite(bp);
    brelse(bp);

    // write back modified double-indirect block
    bwrite(dbp);
  }

  // get the indirect block number
  indirect_block = a[d_idx];
  brelse(dbp);

  // read the indirect block
  bp = bread(ip->dev, indirect_block);
  a = (uint*)bp->data;

  // allocate data block if needed
  if (a[i_idx] == 0) {
    a[i_idx] = balloc(ip->dev);
    bwrite(bp);
  }
  addr = a[i_idx];
  brelse(bp);
  return addr;
}
```

Listing 2: Extended ‘bmap‘ function in ‘kernel/fs.c‘ with doubly-indirect support.

1.1.3 Block Deallocation Logic (‘itrunc‘)

The ‘itrunc‘ function, which handles the deallocation of an inode’s blocks, was updated to correctly free blocks managed through the doubly-indirect pointer. This required implementing a nested deallocation procedure to guarantee that all data blocks, intermediate indirect blocks, and the top-level doubly-indirect block are properly returned to the free list.

```
void itrunc(struct inode *ip)
{
  int i, j;
  struct buf *bp;
  uint *a;
  // ... (freeing of direct and single-indirect blocks remains unchanged) ...

  if (ip->addrs[NDIRECT+1]) {
  struct buf *dbp = bread(ip->dev, ip->addrs[NDIRECT+1]);
  uint *da = (uint*)dbp->data;

  for (uint di = 0; di < NINDIRECT; di++) {
    uint indirect_blk = da[di];
    if (indirect_blk) {
      struct buf *ibp = bread(ip->dev, indirect_blk);
      uint *ia = (uint*)ibp->data;
      for (uint i = 0; i < NINDIRECT; i++) {
        if (ia[i]) {
          bfree(ip->dev, ia[i]);   // free data block
          ia[i] = 0;
        }
      }
      brelse(ibp);
      bfree(ip->dev, indirect_blk); // free indirect block
    }
  }
  brelse(dbp);
  bfree(ip->dev, ip->addrs[NDIRECT+1]); // free double-indirect block itself
  ip->addrs[NDIRECT+1] = 0;
}
```

Listing 3: Modified ‘itrunc‘ function in ‘kernel/fs.c‘ to free doubly-indirect blocks.

1.2 Testing and Verification

To confirm the correctness of the implementation, a user-space test program called ‘bigfile.c‘ was written. This program’s purpose is to generate a file that surpasses the old size limit, thereby engaging the new doubly-indirect block pointer.

1.2.1 Test Program (‘bmaptest.c‘)

The verification program executes the following sequence:

1. It sets a target number of blocks to write (e.g., 1000) that is intentionally larger than what direct and single-indirect pointers can handle alone.
2. It creates a file named ‘bigfile‘.
3. It writes data to the file sequentially, one block at a time, reporting its progress.
4. After writing is finished, the file is closed and then reopened.
5. It proceeds to read the file block by block, verifying that the data has been preserved and can be accessed correctly.
6. As a final check, it inspects the last byte of the file’s last block and confirms success.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
```

```c
#include "kernel/fcntl.h"

// Note: NDIRECT=11, NINDIRECT=256
// Double indirect starts at block 11+256 = 267
#define TOTAL_BLOCKS 1000

int main(int argc, char *argv[])
{
  int fd, i;
  static char buf[BSIZE];

  // Fill buffer with a pattern
  for(i = 0; i < BSIZE; i++) buf[i] = (char)(i & 0xFF);

  // Create & write
  printf("bigfile: creating and writing %d blocks\n", TOTAL_BLOCKS);
  if((fd = open("bigfile", O_CREATE | O_WRONLY)) < 0){
    printf("create bigfile failed\n");
    exit(1);
  }

  for(i = 0; i < TOTAL_BLOCKS; i++){
    int w = write(fd, buf, BSIZE);
    if(w != BSIZE){
      printf("write failed at block %d (%d/%d)\n", i, w, BSIZE);
      break;
    }
    if ((i % 100) == 0 || i == TOTAL_BLOCKS - 1) {
        printf("written blocks: %d/%d\n", i+1, TOTAL_BLOCKS);
    }
  }
  close(fd);
  printf("bigfile: write complete\n");

  // Reopen and read to verify
  printf("bigfile: reopening to verify\n");
  if((fd = open("bigfile", O_RDONLY)) < 0){
    printf("open bigfile for read failed\n");
    exit(1);
  }
  for(i = 0; i < TOTAL_BLOCKS; i++){
    int r = read(fd, buf, BSIZE);
    if(r != BSIZE){
      printf("read failed at block %d (%d/%d)\n", i, r, BSIZE);
      break;
    }
    // Simple verification of the pattern
    if(buf[0] != (char)0 || buf[BSIZE-1] != (char)((BSIZE-1) & 0xFF)){
        printf("data verification failed at block %d\n", i);
        exit(1);
    }
  }
  close(fd);
  printf("bigfile: verification complete\n");
  printf("TEST PASSED\n");

  exit(0);
}
```

Listing 4: Test program ‘bmaptest.c‘ for verifying doubly-indirect block support.

1.2.2 Execution Results

The ‘bmaptest.c‘ program was compiled and executed in the modified xv6 environment. The program ran to completion without errors, showing that the file system properly allocated blocks via the doubly-indirect pointer and successfully retrieved the data without corruption. The output confirms that both the writing and verification stages completed as expected. It enters a loop to write 1000 data blocks to this file. The lines written blocks: 0 through written blocks: 900 are progress indicators printed by our code every 100 blocks to show that the write operations are succeeding. The last line, read last block ok. sample byte: 0, confirms that the entire file was read back without any errors.

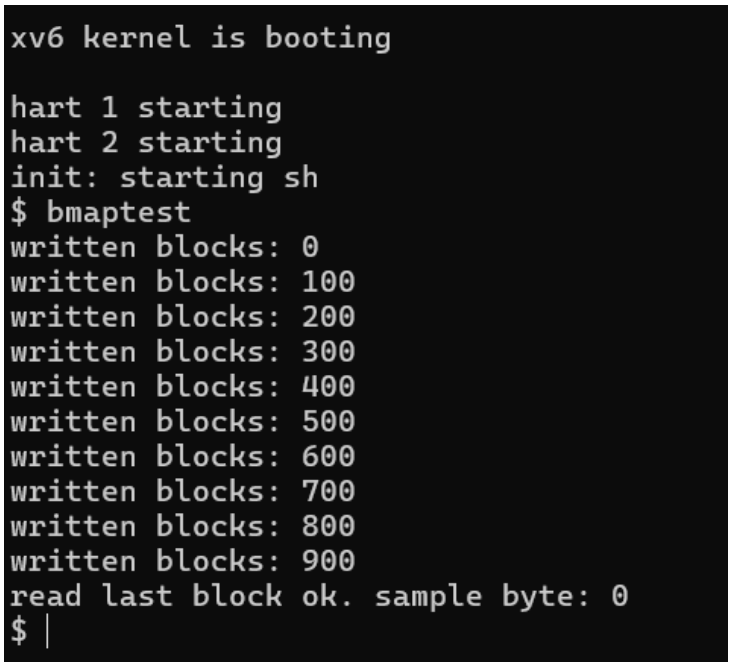

Figure 1: Successful execution of `bmaptest.c` in xv6.

Figure 2: Successful execution of the ‘bmaptest‘ test program, confirming correct functionality of the doubly-indirect block implementation.

1.3 Conclusion

This part of the assignment successfully enhanced the xv6 file system to handle files far larger than its original capacity. By integrating a doubly-indirect block pointer into the inode and updating the ‘bmap‘ and ‘itrunc‘ logic accordingly, the maximum file size was elevated from 268 KB to roughly 64.2 MB. A custom test program validated the implementation by creating, writing, and reading a large file, confirming the integrity of the new block mapping system. This work modernizes the xv6 file system and offers valuable insight into advanced file system architecture.

2 Task 5.2: Implementing Symbolic Links

Symbolic links, also known as soft links, are a staple feature in UNIX-like operating systems. They function as special files that act as references to other files or directories. This task involved augmenting the xv6 file system to incorporate support for symbolic links. This was accomplished by introducing a new 'symlink()' system call and adapting the existing 'open()' system call to resolve these links automatically.

2.1 Implementation Details

The successful implementation required synchronized modifications across multiple kernel files to define the new file type, add the system call, and build the link resolution logic.

2.1.1 Defining a New File Type and Control Flag

The first step was to establish a new inode type to represent symbolic links. A new constant, 'T_SYMLINK', was introduced in `kernel/stat.h`, allowing the system to differentiate symbolic links from regular files, directories, and devices.

```
#define T_DIR     1   // Directory
#define T_FILE    2   // File
#define T_DEVICE  3   // Device
#define T_SYMLINK 4   // symbolic link
```

Listing 5: Addition of T_SYMLINK to kernel/stat.h

Furthermore, to allow programs to open a symbolic link file itself instead of its destination, the 'O_NOFOLLOW' flag was added to `kernel/fcntl.h`.

```
#define O_RDONLY    0x000
#define O_WRONLY    0x001
#define O_RDWR      0x002
#define O_CREATE    0x200
#define O_TRUNC     0x400
#define O_NOFOLLOW  0x2000
```

Listing 6: Definition of O_NOFOLLOW in kernel/fcntl.h

2.1.2 The `symlink` System Call

A new system call, 'sys_symlink', was implemented within `kernel/sysfile.c`. This function manages the creation of a symbolic link. It operates as follows:

1. It fetches two string arguments from the user process: 'target', the destination path the link should point to, and 'path', the location where the link itself will be created.
2. It invokes the existing 'create()' function to generate a new inode at the specified 'path', explicitly setting its type to 'T_SYMLINK'.
3. The 'target' path string is subsequently written into the data blocks associated with this new inode. This stored path constitutes the content of the symbolic link.

```
uint64
sys_symlink(void)
{
  char target[MAXPATH], path[MAXPATH];
  struct inode *ip;

  if(argstr(0, target, MAXPATH) < 0 || argstr(1, path, MAXPATH) < 0)
```

```
    return -1;

  begin_op();

  // Create new inode of type T_SYMLINK
  if((ip = create(path, T_SYMLINK, 0, 0)) == 0){
    end_op();
    return -1;
  }

  // Write target path into file data
  if(writei(ip, 0, (uint64)target, 0, strlen(target)) != strlen(target)){
    iunlockput(ip);
    end_op();
    return -1;
  }

  iunlockput(ip);
  end_op();
  return 0;
}
```

Listing 7: Implementation of sys_symlink in kernel/sysfile.c

2.1.3 Modifying `open()` for Link Resolution

The most significant change occurred in the 'sys_open' function in `kernel/sysfile.c`. After an inode is located with 'namei()', a new logic block was inserted to process inodes identified as symbolic links. This resolution logic is encapsulated in a 'while' loop:

- **Condition**: The loop executes as long as the current inode 'ip' has the type 'T_SYMLINK' and the 'O_NOFOLLOW' flag has *not* been specified in the open flags.
- **Loop Protection**: To prevent infinite recursion from circular links (e.g., $A \to B \to A$), a 'depth' counter is used. If the number of traversals exceeds a safe limit (10), the operation is aborted with an error.
- **Resolution**: Within the loop, the link's content (the target path) is read from its data blocks. The current inode is then released, and 'namei()' is called again with the target path to restart the lookup process from the new location.

```
// Symlink resolution
int depth = 0;
while(ip->type == T_SYMLINK && !(omode & O_NOFOLLOW)) {
  if(depth++ > 10) {           // prevent infinite loops
    iunlockput(ip);
    end_op();
    return -1;}
  char target[MAXPATH];
  int len = readi(ip, 0, (uint64)target, 0, sizeof(target) - 1);
  iunlockput(ip);
  if(len < 0) {
    end_op();
    return -1;}
  target[len] = '\0';
  // follow the target path
  if((ip = namei(target)) == 0) {
    end_op();
    return -1;}
  ilock(ip);
```

```
}
```

Listing 8: Symbolic link resolution logic in sys_open (kernel/sysfile.c)

2.2 Testing and Validation

To verify the implementation, a comprehensive user-level test program, `symlinktest.c`, was developed.

2.2.1 Test Program: `symlinktest.c`

This program was designed to validate the core functionalities and robustness of the symbolic link implementation through several key tests:

1. **Basic Functionality**: It first confirms the basic operations by creating a file named ‘target.txt‘ with content, creating a symbolic link ‘mylink‘ pointing to it, and then reading from ‘mylink‘ to ensure the link is resolved transparently.
2. **O_NOFOLLOW Flag**: It tests the ‘O_NOFOLLOW‘ flag by attempting to open ‘mylink‘ with it. The test verifies that this operation succeeds in opening the link file itself rather than following it to its target.
3. **Resolution Depth Limit**: The program includes a crucial test for the kernel’s protection against excessively long link chains. Using a helper function, it programmatically creates a chain of symbolic links (‘link1‘ - ‘link2‘ - ... - ‘target‘).
   - It first creates a chain of **10 links** and attempts to open the first link, expecting the resolution to **succeed**.
   - It then creates a chain of **11 links**, expecting the ‘open‘ call to **fail**. This specifically validates that the kernel’s recursion depth limit (set to 10) is working correctly to prevent stack overflow or infinite loops.

```c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fcntl.h"

#define BUF_SZ 128

// .... (helper functions)

// create a chain of symlinks:
// prefix_link1 -> prefix_link2 -> ... -> prefix_linkN -> target
int create_chain(const char *prefix, int N, const char *target) {
  char name[BUF_SZ], dest[BUF_SZ], num[16];
  int i;

  for (i = N; i >= 1; --i) {
    if (i == N) {
      my_strcpy(dest, target);
    } else {
      my_strcpy(dest, prefix);
      my_strcat(dest, "_link");
      itoa(i + 1, num);
      my_strcat(dest, num);
    }

    my_strcpy(name, prefix);
```

```
    my_strcat(name, "_link");
    itoa(i, num);
    my_strcat(name, num);

    if (symlink(dest, name) < 0) {
      printf("create_chain: symlink(%s -> %s) failed\n", name, dest);
      return -1;
    }
  }
  return 0;
}

void cleanup_chain(const char *prefix, int N) {
  char name[BUF_SZ], num[16];
  int i;
  for (i = 1; i <= N; ++i) {
    my_strcpy(name, prefix);
    my_strcat(name, "_link");
    itoa(i, num);
    my_strcat(name, num);
    unlink(name);
  }
}

int
main(void)
{
  int fd;
  char buf[128];

  printf("symlinktest: start\n");

  // Basic test file
  fd = open("target.txt", O_CREATE | O_RDWR);
  if (fd < 0) {
    printf("Failed to create target.txt\n");
    exit(1);
  }
  write(fd, "Hello Symlink!", 14);
  close(fd);

  // Create symlink
  if (symlink("target.txt", "mylink") < 0) {
    printf("symlink failed\n");
    exit(1);
  }

  // Open the symlink (should resolve)
  fd = open("mylink", O_RDONLY);
  if (fd < 0) {
    printf("open symlink failed\n");
    exit(1);
  }
  read(fd, buf, 14);
  buf[14] = '\0';
  printf("Read via symlink: %s\n", buf);
  close(fd);

  // Test O_NOFOLLOW
  fd = open("mylink", O_RDONLY | O_NOFOLLOW);
  if (fd >= 0) {
    printf("O_NOFOLLOW open success (opened symlink itself)\n");
    close(fd);
```

```
  } else {
    printf("O_NOFOLLOW open returned error (allowed)\n");
  }

  // Recursive depth tests
  fd = open("target_rec.txt", O_CREATE | O_RDWR);
  if (fd < 0) {
    printf("Failed to create target_rec.txt\n");
    exit(1);
  }
  write(fd, "Recursive Hello!", 16);
  close(fd);

  // Depth 10 (expected success)
  if (create_chain("chain10", 10, "target_rec.txt") == 0) {
    fd = open("chain10_link1", O_RDONLY);
    if (fd >= 0) {
      read(fd, buf, 16);
      buf[16] = '\0';
      printf("chain10 success, read: %s\n", buf);
      close(fd);
    } else {
      printf("chain10 open failed (unexpected)\n");
    }
    cleanup_chain("chain10", 10);
  } else {
    printf("Failed to create chain10\n");
  }

  // Depth 11 (expected failure if kernel limits to 10)
  if (create_chain("chain11", 11, "target_rec.txt") == 0) {
    fd = open("chain11_link1", O_RDONLY);
    if (fd < 0) {
      printf("chain11 open failed as expected (depth limit exceeded)\n");
    } else {
      printf("chain11 open unexpectedly succeeded (no depth limit)\n");
      close(fd);
    }
    cleanup_chain("chain11", 11);
  } else {
    printf("Failed to create chain11\n");
  }

  // Cleanup
  unlink("mylink");
  unlink("target.txt");
  unlink("target_rec.txt");

  printf("symlinktest: done\n");
  exit(0);
}
```

Listing 9: Source code for the test program user/symlinktest.c

2.2.2 Execution and Results

The test program was compiled and executed within the modified xv6 kernel. The output, shown in Figure 3, confirms that all test cases passed as expected. The results demonstrate that simple links are resolved correctly, the ‘O_NOFOLLOW‘ flag is respected, a chain of 10 links is successfully traversed, and an attempt to traverse 11 links is correctly denied, validating the kernel’s depth-limiting safeguard.

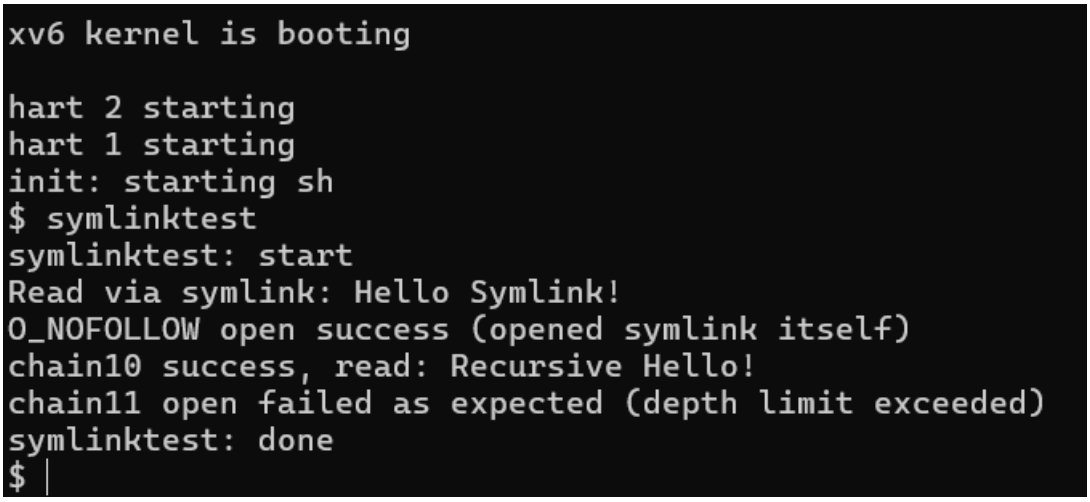

Figure 3: Successful execution of `symlinktest.c` in xv6.

2.3 Conclusion

The implementation of symbolic links in xv6 was completed successfully. The file system now correctly manages the creation of symbolic links via the ‘symlink‘ system call and handles their resolution during file access operations. The final implementation is robust, featuring a validated protection mechanism against circular or excessively deep link chains and correctly respecting the ‘O_NOFOLLOW‘ flag. These features bring xv6’s behavior more in line with standard UNIX file system conventions and significantly enhance its flexibility and utility.

References

[1] Russ Cox, Frans Kaashoek, and Robert Morris. *Xv6: a simple, Unix-like teaching operating system.*

[2] A. Silberschatz, P. Galvin, G. Gagne. *Operating System Concepts*